

Wireshark

1. Introduction

This lab introduces packet capture (packet sniffing) and network traffic analysis with the Wireshark tool, and network reconnaissance and scanning with `nmap`. Where the introductory version of this lab asks you to **find** things in a trace, this version asks you to **explain why** the trace looks the way it does*, and in several places to **correlate two tools against each other** — run a scan with `nmap` while Wireshark is capturing, and use the capture to verify, byte-for-byte, what `nmap` actually did on the wire. Your guide has asked for a difficult assignment; the difficulty here is depth, not just volume of questions.

2. Objective of the Lab

In this laboratory, four main observations are required to investigate, with details in the following.

Observation 1: A packet sniffer is an application which can capture and analyze network traffic passing through a system's Network Interface Card (NIC). The sniffer sets the card to promiscuous mode, meaning all traffic is read, whether addressed to that machine or not. Wireshark (formerly **ethereal**) is the standard GUI packet sniffer/analyzer used in this lab; `tshark` is its command-line counterpart, used later in this lab for anything that needs to be repeated, automated, or run headless.

Observation 2: Packet analysis. Wireshark is an open-source, cross-platform packet capture and analysis tool. The GUI gives a detailed breakdown of the network protocol stack for each packet, colorizes packet details by protocol, and lets you filter, search, and follow TCP streams. Wireshark can save captures to disk for offline analysis and can generate statistics over a capture file. It is used for network troubleshooting, security investigation, and protocol study — and, because it decodes traffic faithfully, it will happily show you a plaintext credential passed by an unencrypted protocol (Telnet, FTP, SNMP, POP, HTTP) exactly as it crossed the wire.

Observation 3: Network scanning is done at the reconnaissance stage of a structured attack. A scanner such as `nmap` tells an attacker which remote machines are alive and reachable, and which services they run. Scanning includes host discovery (sweeps), port scans, and OS/service fingerprinting. Different `scan *types*` (`-sS``, `-sT``, `-sU``, `-sA``, `-sN``, ...) are not interchangeable — they produce genuinely different packets on the wire, which is exactly what Part III of this lab makes you prove. Defensively: only run the services you need, and use ``netstat`/`ss`` to audit what is actually listening on a host you're responsible for.

Observation 4: Sniffer and scanner are the two ends of the same reconnaissance problem. A scanner tells an attacker what a target *offers*; a sniffer tells an attacker (or a defender) what is *actually happening* on a link. This lab deliberately makes you run both, at the same time, against each other, so that you stop treating "what a tool reports" and "what the wire actually carried" as the same thing — they frequently aren't, and the gap between them is where a lot of real detection work lives.

3. Laboratory setup

- The lab machines should have both Linux and Windows available (dual-boot or VM). Install the latest Wireshark and on whichever OS you use.

- You will need two hosts that can reach each other for Part III () and several of the Security Assignment tasks — two VMs on VirtualBox/VMware on a Host-only or Internal network is sufficient; you do not need physical hardware.

- Authorization boundary, stated plainly: every scan and capture in this lab must be run only between machines you control (your own VMs, your own host, or a target explicitly provided for this course). Running or a sniffer against any other host on the campus network, or any host you do not own or have explicit permission to test, is out of scope for this lab and is a policy violation, not just bad practice — see Part IV before you touch a real network.

4. Assignments

(i) Networking Assignment — IP, ICMP, TCP, UDP

Capture your own traffic (ping a host, browse a page that transfers a reasonably sized file, and generate at least one UDP exchange such as a DNS query) and answer the following directly from your capture — do not answer from memory or from the textbook where the question asks you to determine something from the trace.

A. ICMP / IP

1. Select the first ICMP Echo Request your computer sends. Expand the IP layer. What is your computer's IP address, and what is the value in the upper-layer protocol field?

2. How many bytes are in the IP header, and how many in the payload? Show how you computed the payload size from the fields visible in Wireshark (do not just read a "length" field and stop — show the arithmetic).

3. Has this datagram been fragmented? State exactly which flag/field you checked, and what its value would need to be for the answer to be "yes."

4. Across the whole series of ICMP Echo Request/Reply pairs you captured: which IP header fields change every time, which stay constant, and — separately — which of those **must** stay constant and which **must** change for the protocol to function correctly? Justify each "must."

5. Describe the pattern in the Identification field across your ICMP series. Is it sequential, random, or something else? What does that tell you about how your OS generates it?

6. Now generate an ICMP TTL-exceeded case (e.g. via ``traceroute`/`tracert`` to a distant host, or by crafting one with ``hping3`/`scapy`` if available). For the TTL-exceeded replies from the first-hop router: do the Identification and TTL fields of the **replies** stay constant across the series, or do they vary? Explain why, tying your answer to the fact that these replies are generated by the router, not forwarded from your original packets.

B. TCP — handshake and reliability

Capture a file transfer or a reasonably large HTTP(S) download so you have a multi-segment TCP connection to work with.

7. What IP address and TCP port does your client use as the connection's source? What is the sequence number of the SYN segment that opens the connection, and what specifically in the segment marks it as a SYN (not just "the flags say so" — name the exact flag bit)?

8. What is the sequence number of the SYN-ACK, and what is the acknowledgment number? Show the arithmetic by which the server derived that acknowledgment number from your SYN.

9. Treating the first data-carrying segment as segment 1, list the sequence numbers of the first six segments of the connection, their send times, and the time each corresponding ACK arrived. Compute the RTT for each of the six. From these six samples, compute an EWMA-style EstimatedRTT (state the alpha you used).

10. What is the length of each of those first six segments? Is the length constant, and if not, why not (think about MSS and how much application data was available to send at each point)?

11. What is the smallest receive-window value advertised by the receiver anywhere in the trace? Does the sender's send rate visibly drop in response to a small advertised window at any point, or does congestion control (not flow control) appear to be the limiting factor instead? How can you tell the two apart from the trace?

12. Are there any retransmitted segments? State precisely what you checked for (Wireshark's own "TCP Retransmission" flag is one way — also check by hand: same sequence number appearing twice with no acknowledgment in between) and report how many you found.

13. Find one case of a duplicate ACK (the same ACK number appearing on more than one incoming segment). Does it trigger a fast retransmit (three duplicate ACKs followed by a retransmission before a timeout), or is it an isolated duplicate with no retransmission? Show the packet numbers for your evidence either way.
14. How much data does the receiver typically acknowledge per ACK — does it look like it's acknowledging every segment, or does it appear to be using delayed ACKs (acknowledging roughly every other segment)? Show two or three consecutive examples.
15. Compute the throughput (bytes transferred per unit time) for the whole connection. Show your formula and the numbers you plugged into it.

C. UDP

16. Select one UDP packet from your trace. Without consulting a textbook, determine directly from the packet how many fields are in the UDP header, and name them.
17. From the packet content field, determine the length in bytes of each UDP header field.
18. What exactly does the Length field measure — the whole datagram, just the payload, or something else? Verify your answer against your captured packet's actual byte count.
19. Given your answer to Q18, what is the theoretical maximum number of bytes in a UDP payload? Show the arithmetic (hint: it follows from the field width you found in Q17).
20. What is the largest possible UDP source port number, and why (tie it to a field width you already measured)?
21. What is the IP protocol number for UDP? Give it in both hex and decimal, and state exactly which field of the IP header you read it from.
22. Find a pair of UDP packets where your host sends the first and receives a reply as the second. Describe the relationship between the two packets' port numbers, and explain in one sentence why a reply **must** target the original sender's source port for the sending application to ever see it.

(ii) Security Assignment — Filters and Detection

23. List two or three real network problems from your own experience (or a plausible scenario) where Wireshark would be the right tool to diagnose them. Be specific about what symptom you'd be chasing (e.g. "intermittent slow page loads" is not specific; "TCP retransmissions climbing on one particular route" is).
24. Build the following display filters and test each against a capture that actually contains matching traffic. For each, record the exact filter string and a one-line note on what it isolates:

- ARP packets from a specific source MAC address (build this one **interactively** in Wireshark first — right-click a field, "Apply as Filter" — then write down the resulting filter string it generated, e.g. ``arp.src.hw_mac == xx:xx:xx:xx:xx:xx``).

- A compound filter combining an IP source, an IP destination, and a protocol condition with ``&&`/`||`` (e.g. ``ip.dst==A && ip.src==B || tcp.port==C``) — and in ``REPORT.md``-style notes, explain what operator precedence means here: does your filter mean what you think it means, or do you need explicit parentheses?

- DHCP request/response traffic specifically for the moment a new machine joins the network (``bootp`/`dhcp`` filter — capture this by connecting a fresh VM or renewing a lease with ``dhclient -r && dhclient``).

- TCP RST packets only.

- HTTP responses with status code ≥ 400 .

- A filter for packets above a size threshold you choose, and a one-line note on why that threshold might matter for detecting something (e.g. unusually large DNS responses).

25. Promiscuous vs. non-promiscuous capture. Perform two captures on the same interface over the same rough time window — one with promiscuous mode on, one off. Find at least one packet present in the promiscuous capture that is absent from the non-promiscuous one. What specifically about that packet (source/destination MAC or IP relative to your own machine) tells you it would only be visible in promiscuous mode?

26. Is there a single display filter that would let you exclude localhost traffic as either source or destination in one shot? Write it, and explain why you might want to exclude localhost when hunting for external reconnaissance activity.

27. Locate ARP traffic in a capture and confirm Wireshark applied the correct dissector to it (i.e., it's actually parsed as ARP, not shown as raw/unknown bytes). If you have access to a capture containing an ARP spoofing attempt (from a provided sample, or one you generate yourself in your own isolated VM lab under the authorization boundary in Section 3), identify it by finding conflicting IP-to-MAC mappings — the same IP address claimed by two different MAC addresses in different ARP replies. Give the exact packet numbers of the conflicting replies.

28. Sniff across three different virtual network types — Host-only, NAT, and Bridged/Internal — using two VMs. For each network type, record: can VM1 ping VM2? Can either ping the host? If you run Wireshark on the **host** while the two VMs talk to each other, do you see

their traffic at all in each mode? Explain the visibility difference across the three modes in terms of where traffic is actually switched/routed.

29. Capture some encrypted Wi-Fi or TLS traffic. What can you still determine from it **despite** the encryption (e.g., SNI in the TLS ClientHello, packet sizes/timing, destination IPs), and what is genuinely hidden from you? This distinction — metadata vs. content — is the practical reason traffic analysis remains useful even against encrypted traffic.

30. Diagnosing an attack, with evidence. Using a provided/downloaded sample capture containing a credential-harvesting or MITM scenario (or one you generate yourself, under the Section 3 authorization boundary, using a lab tool such as an ARP MITM setup against your **own** isolated VMs only): identify the plaintext credential that was exposed, name the exact protocol that carried it in plaintext, and give the packet number and the "Follow Stream" output line where the credential appears. Do not paste real, currently valid credentials into your report if you generated the capture using your own throwaway lab account — use a clearly fake placeholder account for the capture in the first place.

Submission

Submit the following:

1. PCAPNG file (`.pcapng`) — Submit the packet capture file(s) generated during the lab.

2. A document (`.pdf`) — Submit a document containing:

- Answers to all the lettered/numbered questions.
- Relevant screenshots from Wireshark for each answer where evidence is required.
- A brief explanation below each screenshot describing what the screenshot shows, why it is relevant, and how it supports the answer.
- A short statement of your understanding/objective of each important screenshot, explaining what you were trying to observe or verify rather than simply pasting the screenshot.